Mississippi Valley State University

CS 341 — Discrete Structures

Lecture 48: Halving, Not Decrementing

Divide-and-Conquer Recurrences and the Master Theorem

MCS Sections 21.4 and 21.5

Name: _____ Date: _____

Where we are

Wednesday's method handles recurrences that step back a fixed amount: $f(n-1)$, $f(n-2)$. Merge sort does something else — it reaches $f(n/2)$, halfway back — and the characteristic equation has nothing to say about it.
Today: the tool for that shape, and the reason merge sort beats Towers of Hanoi by an astronomical margin despite having a nearly identical-looking equation.

By the end of today you should be able to

1. Recognise a divide-and-conquer recurrence.
2. Apply the Master Theorem and say which case you are in.
3. Explain why subproblem *size* matters more than work per call.

1. The shape

Divide-and-conquer recurrence

$$T(n) = a\,T\left(\frac{n}{b}\right) + g(n)$$

where $a \geq 1$ subproblems are solved, each of size n/b with $b > 1$, and $g(n)$ is the work to split and recombine.

Recurrence	a	b	$g(n)$
Merge sort: $T(n) = 2T(n/2) + n - 1$	2	2	$n-1$
Binary search: $T(n) = T(n/2) + 1$	1	2	1

2. The Master Theorem

Everything turns on comparing $g(n)$ with $n^{\log_b a}$ — the amount of work the recursion alone generates.

Theorem 21.4.2 — Master Theorem

For $T(n) = a\,T(n/b) + g(n)$, let $p = \log_b a$.

- **Case 1.** If $g(n)$ grows *slower* than n^p, then $T(n) = \Theta(n^p)$.
- **Case 2.** If $g(n) = \Theta(n^p)$, then $T(n) = \Theta(n^p \log n)$.
- **Case 3.** If $g(n)$ grows *faster* than n^p (and a mild regularity condition holds), then $T(n) = \Theta(g(n))$.

Read it as a competition. The recursion contributes n^p and the combining step contributes $g(n)$. Whichever is larger wins. If they tie, the answer picks up a factor of $\log n$.

3. Four applications

Recurrence	p	Case	Answer
$T(n) = 2T(n/2) + n$	1	2	$\Theta(n \log n)$ — merge sort
$T(n) = T(n/2) + 1$	0	2	$\Theta(\log n)$ — binary search
$T(n) = 4T(n/2) + n$	2	1	$\Theta(n^2)$
$T(n) = 2T(n/2) + n^2$	1	3	$\Theta(n^2)$

Two results you already had

Row one is merge sort's $\Theta(n \log n)$, which Lecture 45 read off a diagram. Row two is binary search's $\Theta(\log n)$, which Lecture 41 got from the height of a binary tree.
The Master Theorem derives both from the equation alone — no picture, no tree.

Try one. $T(n) = 3T(n/2) + n$. Here $p = \log_2 3 \approx 1.585$, and $g(n) = n$ grows slower, so Case 1 gives $T(n) = \Theta(n^{1.585})$ — worse than merge sort, better than n^2.

4. Which matters more: how many, or how much?

Four recurrences we have solved:

Problem	Recurrence	Solution
Towers of Hanoi	$T_n = 2T_{n-1} + 1$	$\sim 2^n$
Merge sort	$T_n = 2T_{n/2} + n - 1$	$\sim n \log n$
Hanoi variant	$T_n = 2T_{n-1} + n$	$\sim 2 \cdot 2^n$
Fibonacci	$T_n = T_{n-1} + T_{n-2}$	$\sim 1.618^n$

The lesson of the table

Hanoi and merge sort look almost identical — both split into *two* subproblems. But Hanoi's subproblems have size $n-1$ (barely smaller) with 1 unit of extra work, while merge sort's have size $n/2$ (much smaller) with n units of extra work.

Merge sort does *far more* work per call and is still faster by an astronomical margin. Sorting 64 items takes a few hundred comparisons; moving 64 disks takes more than 10^{19} steps.

Shrinking the subproblem matters enormously; the work per call barely matters. Compare rows one and three: raising the per-call cost from 1 to n only doubled the answer.

And Fibonacci, for contrast

Fibonacci's subproblems are size $n-1$ and $n-2$ — only slightly smaller than Hanoi's — yet its solution is exponentially *smaller*, 1.618^n against 2^n. Having one subproblem instead of two is worth more than any amount of extra work per call.

That is why computing Fibonacci numbers by naive recursion is slow, and why memoising it is the standard first example of dynamic programming.

Exit ticket

1. Apply the Master Theorem to $T(n) = 2T(n/4) + n$. Give p, the case, and the answer.

2. In one sentence: why does merge sort beat Towers of Hanoi despite doing more work per recursive call?

Before next class

Read *MCS* Section 21.5, "A Feel for Recurrences," p. 888 — two pages, and the best summary of this block.

Coming next

Graph algorithms: depth-first and breadth-first search, and minimum spanning trees. Both will be analysed with the tools from these two blocks.

Questions I still have